建設CAD対応設計 — 土木PDF→建築CAD→全自動積算

> **ステータス**: 設計完了・未実装（PMF検証後に着手）
> **目的**: 建築積算（CAD図面→仕上表→数量→単価→内訳書）を自動化し、Advanを超える
> **前提**: 現行の土木PDF積算パイプライン（estimator.py）は維持。CADは追加レイヤー
> **関連設計**: `e_01_建設業BPO設計.md` Section 1, `b_04_製造業見積エンジン3層設計.md`

---

0. エグゼクティブサマリー

現状

```
シャチョツー積算AI:
  ・土木の設計内訳書PDF → テキスト抽出 → 数量構造化 → 単価推定 → 内訳書
  ・実績: 15件PDF、356件数量自動抽出、工種分類精度96.3%
  ・制約: 建築CAD図面は非対応

パイロット企業の現実:
  ・総合建設会社は土木と建築の両方を受注
  ・「土木だけ」では積算AIの価値が半減
```

Advan vs シャチョツー — 6ステップ自動化比較

ステップ	Advan Neo仕上	シャチョツー（Phase 3目標）
① 図面読み取り	人間がJW-CAD起動	**CADパース + Vision AI**
② 数量計算	クリック→自動	**全部屋一括自動**
③ 仕上選定	**人間が選択**	**学習型AI提案**
④ 単価適用	**人間が入力**	**単価DB + AI推定**
⑤ 諸経費計算	**なし**	**自動（積算基準準拠）**
⑥ 内訳書生成	Excel出力	**Excel/PDF自動**
自動化率	**2/6**	**6/6**

1. 3フェーズ計画

Phase 1: CADファイル→PDF変換→既存パイプライン（PMF後 +2週）

```
目的: 建築の入口だけ作る。「CAD受け付けられます」と言える最小限
実装:
  □ ユーザーにDXF出力を依頼するガイドUI
  □ DXFアップロード → テキスト抽出 → 既存パイプラインに流す
  □ project_type=building の分岐ロジック
工数: 2週間
自動化率: 3/6（①②は部分的、④⑤⑥は既存流用）
```

Phase 2: DXF直接パース→幾何計算（PMF後 +6週）

```
目的: Advanの②（面積自動計算）を超える。クリック不要
実装:
  □ DXFパーサー（ezdxf）→ レイヤー/エンティティ抽出
  □ 閉じたポリラインから部屋検出（Shoelace formula）
  □ テキストラベルで部屋名自動分類
  □ 仕上マスタデータ（640パターン）+ 手動選定UI
  □ building_estimation_pipeline.py（10ステップ）
工数: 4週間（2名）
自動化率: 4/6（③仕上選定のみ人間）
```

Phase 3: Vision AI + 学習型仕上選定（PMF後 +12週）

```
目的: ③仕上選定をAIで自動化。Advanを完全凌駕
実装:
  □ Vision AIで図面画像から部屋認識（スキャンPDF対応）
  □ フィードバック学習ループ（finish_patterns集約）
  □ confidence付きAI仕上提案UI
  □ Trust Scorer統合
工数: 6週間（2名）
自動化率: 6/6
```

2. 仕上選定の学習ループ（核心の差別化設計）

なぜAdvanが自動化できないか

```
同じ「事務室」でも:
  ・オフィスビル → OAフロア+TC
  ・学校 → 長尺シート
  ・病院 → 長尺シート（耐薬品性）
 → ルールベースでは対応しきれない。しかし「パターン」は存在する
```

学習フェーズ

```
Phase A（0〜10社）: 仕上選定100%人間。選択結果を蓄積
Phase B（10〜50社）: AI提案デフォルト表示。人間は例外のみ修正
  「事務室の床 → OAフロア: 78%」→ 提案。採用率追跡
Phase C（50〜200社）: confidence > 0.85 は自動確定。0.6–0.85は確認。< 0.6は人間
Phase D（200社〜）: 顧客固有パターン + 設計者パターン + グレード整合性チェック
```

Confidenceスコア（製造業3層エンジンと同じ思想）

ソース	Confidence
LLMデフォルト（データなし）	0.3-0.5
統計パターン（finish_patterns集約）	0.6-0.8
顧客固有パターン（3件以上一致）	0.85-0.95

---

3. 技術アーキテクチャ

対応ファイル形式

形式	Phase	パース手法
DXF	Phase 2	`ezdxf` で直接パース
JW-CAD (.jww)	Phase 1: ユーザーにDXF出力依頼 / Phase 2+: 自動変換	
DWG	Phase 2	ODA File Converter → DXF
PDF（ベクター）	Phase 1	テキスト抽出 → 既存パイプライン
PDF（スキャン）	Phase 3	Document AI OCR → Vision AI

ディレクトリ構成（追加分）

```
workers/bpo/construction/
  ├── estimator.py                  # 既存（変更なし）
  ├── cad/                          # ★新規
  │   ├── dxf_parser.py             #   DXFパーサー
  │   ├── geometry.py               #   面積/周長計算（Shoelace formula）
  │   ├── room_detector.py          #   閉領域→部屋検出
  │   └── vision_recognizer.py      #   Vision AI（Phase 3）
  ├── finish/                       # ★新規
  │   ├── selector.py               #   仕上選定エンジン（学習型）
  │   ├── master_data.py            #   仕上材料マスタ
  │   └── learning.py               #   フィードバック学習
  └── building_estimator.py         # ★新規: 建築積算パイプライン
```

建築積算パイプライン（10ステップ）

```
Step 1:  file_converter        .jww/.dwg → .dxf変換
Step 2:  cad_parser            DXFパース → レイヤー/エンティティ
Step 3:  room_detector         閉領域検出 → 部屋リスト（面積/周長）
Step 4:  room_classifier       部屋名認識（テキスト or Vision AI）
Step 5:  finish_selector       仕上選定（学習型AI提案 or 人間選択）
Step 6:  quantity_calculator    仕上×面積 → 数量集計
Step 7:  unit_price_lookup     単価照合（★既存パイプライン再利用）
Step 8:  overhead_calculator   諸経費計算（★既存パイプライン再利用）
Step 9:  breakdown_generator   仕上表 + 内訳書生成
Step 10: output_validator      必須記載事項チェック（★既存再利用）
```

DBスキーマ（追加テーブル）

```
-- CAD図面ファイル管理
CREATE TABLE cad_drawings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID REFERENCES estimation_projects(id),
  company_id UUID REFERENCES companies(id),
  file_format TEXT NOT NULL,     -- jww / dxf / dwg / pdf_vector
  file_url TEXT NOT NULL,
  room_count INTEGER,
  parse_status TEXT DEFAULT 'pending'
);

-- 部屋データ
CREATE TABLE cad_rooms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  drawing_id UUID REFERENCES cad_drawings(id),
  company_id UUID REFERENCES companies(id),
  room_name TEXT,
  room_type TEXT,                -- office / meeting / corridor / toilet
  area_m2 DECIMAL(10,3),
  perimeter_m DECIMAL(10,3),
  ceiling_height_m DECIMAL(5,3) DEFAULT 2.7,
  detection_confidence DECIMAL(3,2)
);

-- 仕上データ（部屋×部位）
CREATE TABLE room_finishes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID REFERENCES cad_rooms(id),
  company_id UUID REFERENCES companies(id),
  part TEXT NOT NULL,             -- floor / wall / ceiling / baseboard
  finish_name TEXT NOT NULL,
  quantity DECIMAL(10,3),
  unit TEXT NOT NULL,             -- m2 / m
  unit_price DECIMAL(15,2),
  selection_method TEXT,          -- manual / ai_suggested / ai_auto
  ai_confidence DECIMAL(3,2)
);

-- 仕上パターン学習データ（匿名集約）
CREATE TABLE finish_patterns (
  room_type TEXT NOT NULL,
  building_type TEXT NOT NULL,
  part TEXT NOT NULL,
  finish_name TEXT NOT NULL,
  usage_count INTEGER DEFAULT 1,
  usage_rate DECIMAL(5,4),
  UNIQUE(room_type, building_type, part, finish_name)
);
```

4. 初期仕上マスタ（640パターン）

```
20用途 × 8建物種別 × 4部位 = 640パターン

代表例:
  事務室×オフィスビル: {床: OAフロア+TC, 壁: PB二重+EP, 天井: 岩綿吸音板}
  会議室×オフィスビル: {床: TC, 壁: PB+VP, 天井: 岩綿吸音板}
  廊下×共通:           {床: 長尺シート, 壁: PB+EP, 天井: 岩綿吸音板}
  便所×共通:           {床: タイル, 壁: タイル腰壁+EP, 天井: ケイカル板+EP}
  教室×学校:           {床: 長尺シート, 壁: PB+EP, 天井: 岩綿吸音板}
  病室×病院:           {床: 長尺シート, 壁: PB+VP, 天井: 化粧石膏ボード}

ソース: 公共建築工事標準仕様書（国交省）
```

5. 後方互換

```
原則: 既存のestimator.py / estimation_pipeline.pyは一切変更しない

・building_estimation_pipeline.py を追加するだけ
・ルーターで project_type に応じて分岐
  civil → 土木パイプライン（既存）
  building → 建築パイプライン（新規）
・estimation_projects / estimation_items テーブルは共用
・単価推定 / 諸経費計算のマイクロエージェントは再利用
```

6. コスト・リスク

開発コスト

Phase	工数	期間
Phase 1	2人週	2週間
Phase 2	8人週	4週間
Phase 3	12人週	6週間
合計	**22人週**	**12週間**

ランニングコスト（Phase 3、1テナントあたり）

項目	月額
Vision AI	¥500-2,000
DXFパース	¥0（自前計算）
ストレージ	¥100-500
合計	**¥700-3,000/月**（BPO月額¥250,000の1%未満）

主要リスク

リスク	対策
JW-CAD直接パースが困難	Phase 1はユーザーにDXF出力を依頼
DXF部屋検出精度	閉じたポリラインのみ。結果は人間確認
仕上パターンの蓄積が遅い	640パターンを初期データとして投入
CADレイヤー名がバラバラ	よくあるパターンリスト + ユーザーマッピングUI